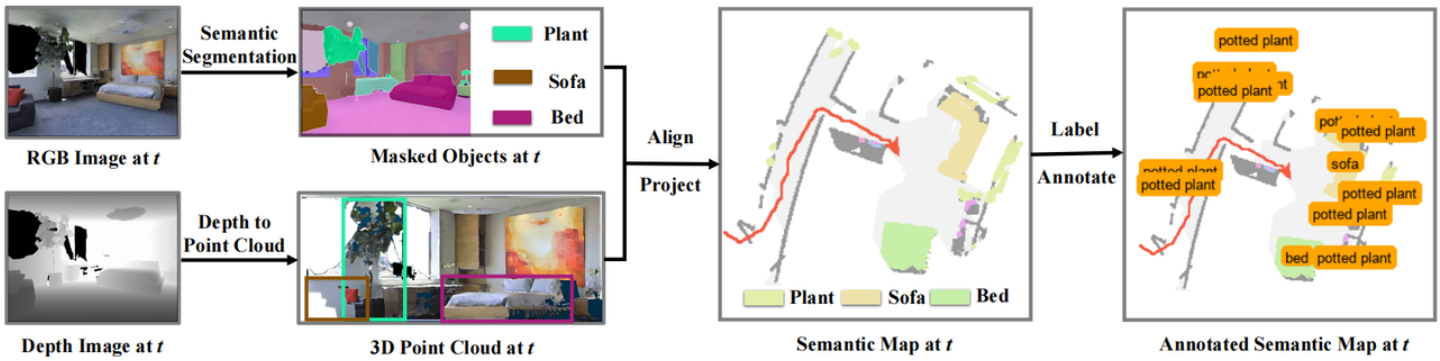
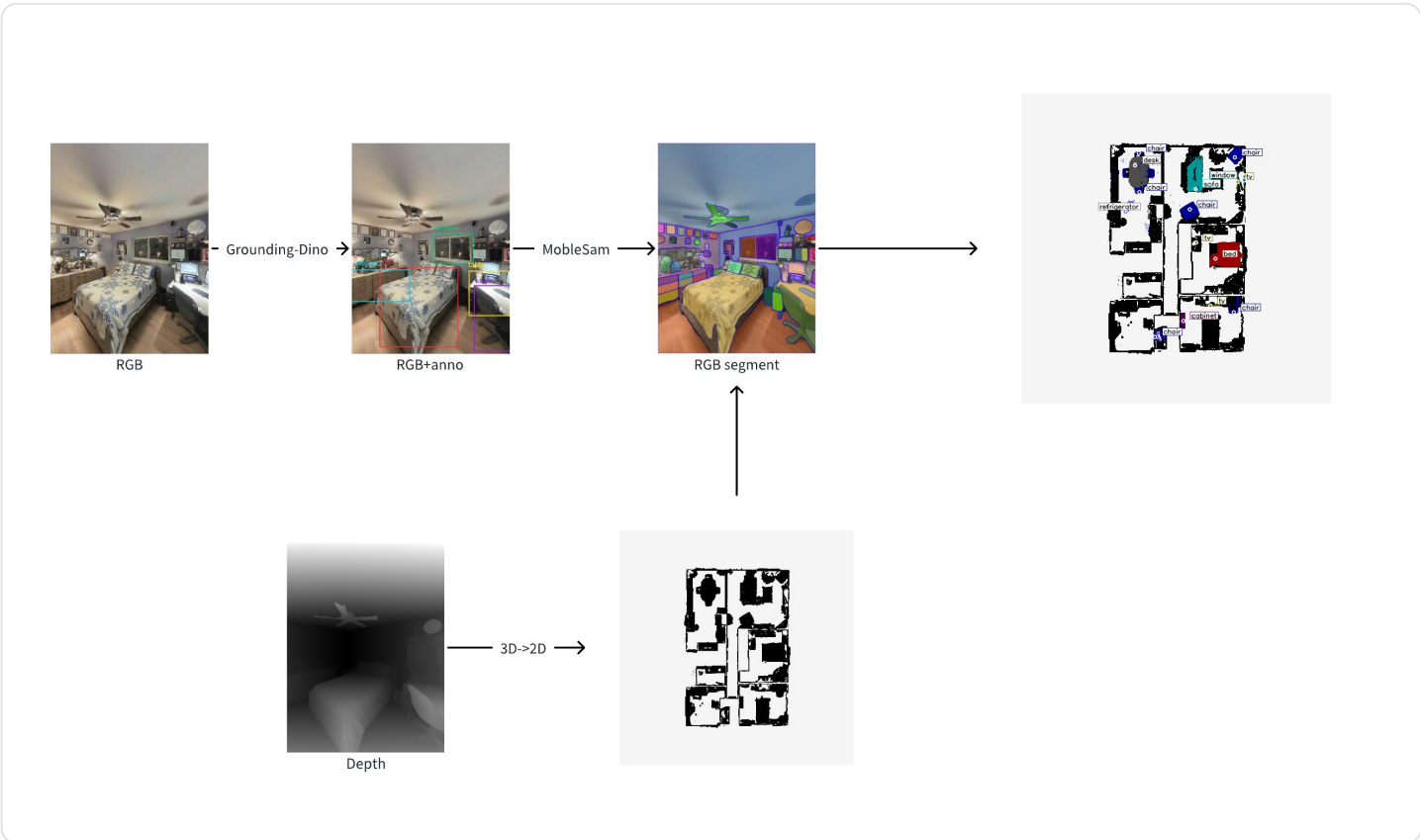
ASM建图架构

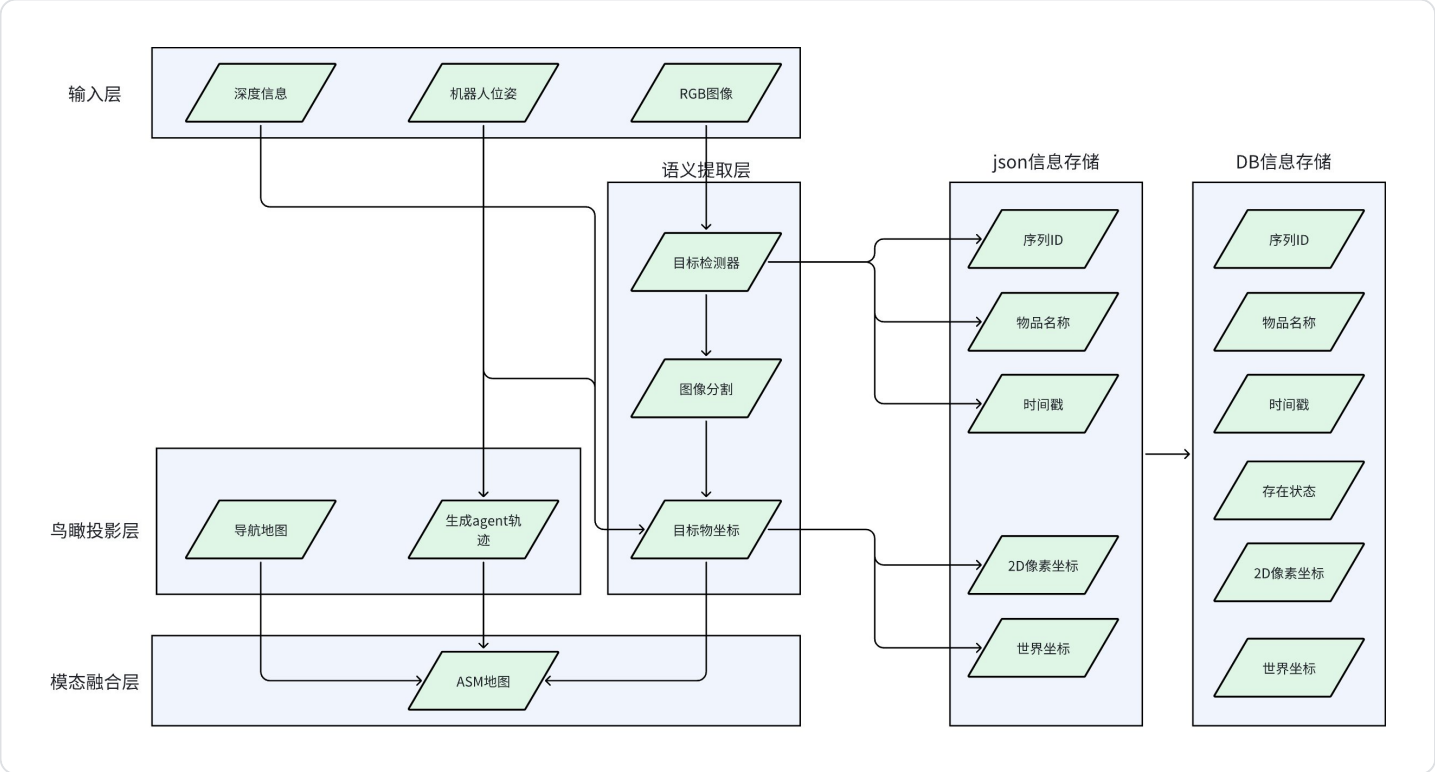
一、原MapNav框架



二、当前ASM地图构建方案



三、ASM架构流程图



RGB数据处理流程

1.1 对象检测阶段

detect_and_update_objects() :

```
RGB图像 → 对象检测器（dino） → 检测结果
      ↓
获取检测框和语义标签
      ↓
对每个检测对象进行分割（moblesam）
      ↓
生成对象掩码(mask)
```

1.2 语义信息提取

- **检测内容**: 物体类别、置信度、边界框
- **分割处理**: 为每个检测对象生成掩码
- **语义标注**: 提取物体名称和属性信息

1.3 语义地图更新

ValueMap.add_object() :

```
对象信息（像素坐标） + 相机内参 + 位姿（外参）
```

```
        ↓
将2D掩码投影到3D空间
        ↓
转换到地图坐标系
        ↓
更新语义地图的对应区域
        ↓
记录对象位置和属性
```

1.4 检测结果保存

- **JSON文件**: 保存检测的对象信息、位置、置信度
- **标注图像**: 保存带检测框的RGB图像
- **语义数据**: 保存对象在地图中的分布信息

深度数据处理流程

2.1深度数据处理

update_map() 方法中的深度处理:

```
原始深度图像
        ↓
深度值裁剪
        ↓
深度数据验证 (有效像素比例检查)
        ↓
深度归一化 (除以最大深度值)
        ↓
转换为ObstacleMap
```

2.2 几何信息提取

- **有效深度检测**: 过滤掉无效或超出范围的深度值
- **相机参数应用**: 进行坐标转换
- **位姿变换**: 将相机坐标系转换到世界坐标系

2.3 障碍物地图更新

ObstacleMap.update_map() 流程:

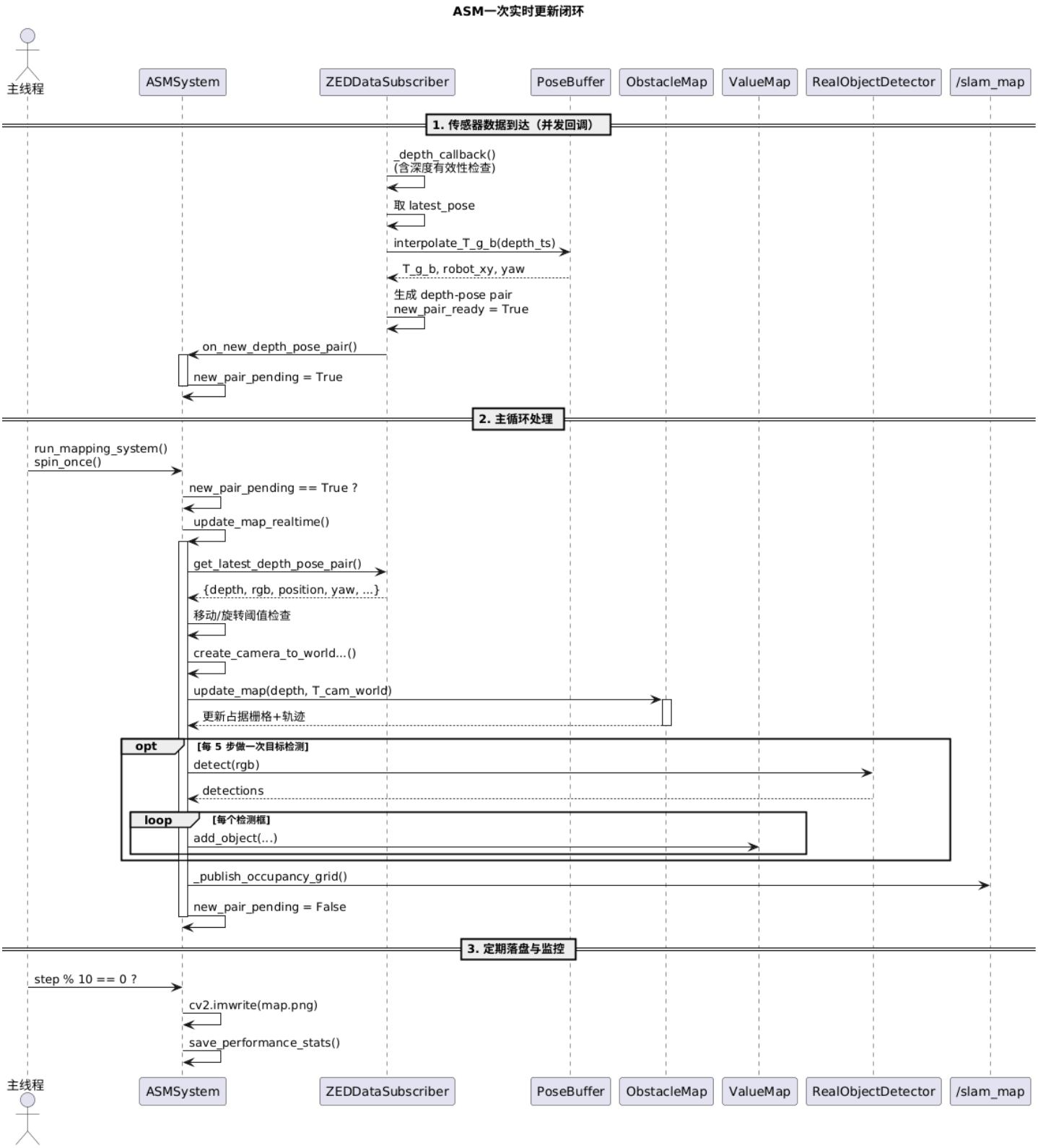
```
深度图像 + 相机位姿
        ↓
```

```
深度值转换为3D点云
      ↓
应用相机内参和外参变换
      ↓
投影到鸟瞰图坐标系
      ↓
更新障碍物占用栅格
      ↓
更新可通行区域信息
```

2.4 轨迹记录

- **Agent轨迹**: 记录扫描过程中的移动路径

三、软件时序图



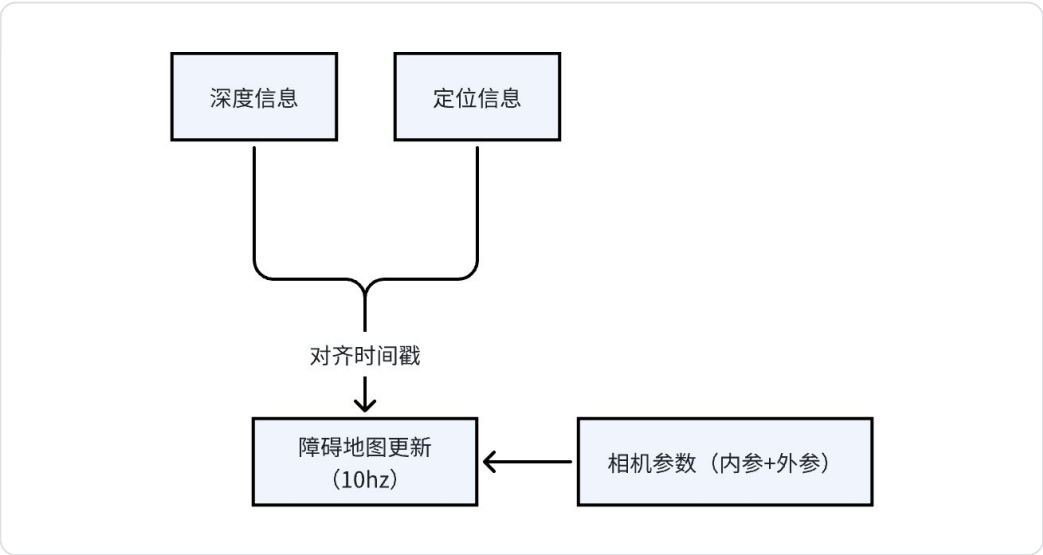
四、当前存在的问题

1. 深度信息3D->2D后存在毛边，与仿真构建仍存在差距；
2. dino识别效果在实机环境中表现存在不足（较小物品）；
3. 动态地图更新功能需开发；

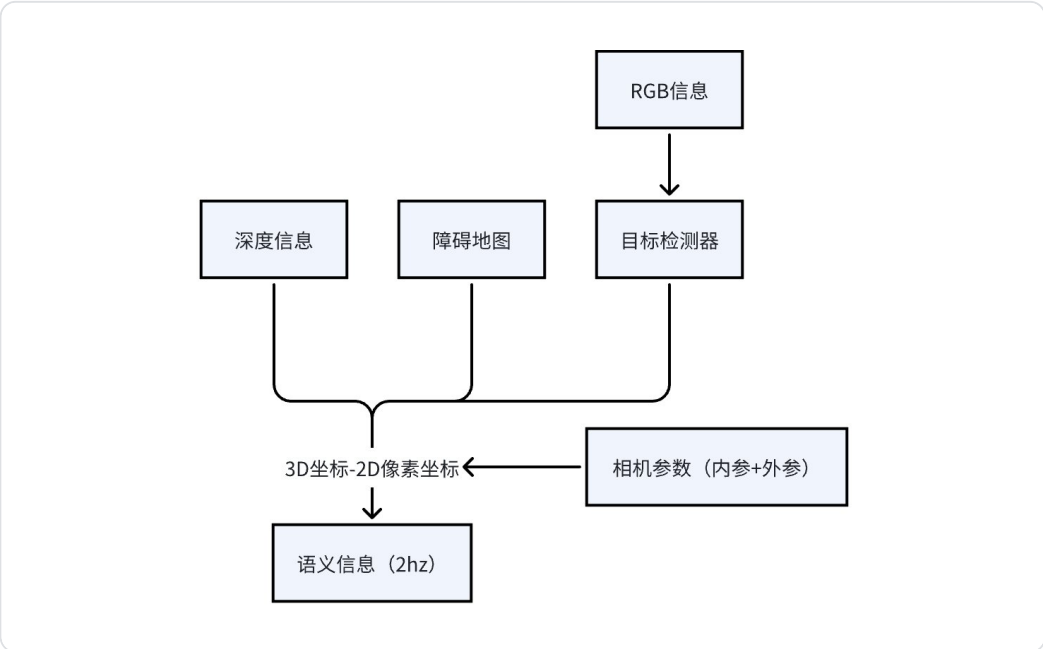
五、动态地图更新方案设计

当前现状：当前ASM地图在更新过程中仅可在首次构建时进行障碍地图与语义地图部分的更新（并不会抹去过去的信息），并未考虑后续若场景发生较大变化造成障碍地图与语义地图发生变化后原先ASM地图无法适配后续任务的情况；

障碍地图



语义信息



当前构思的方法都默认当前已完成一次全屋建图

1. 基于agent半径的地图更新

   把“传感器最大可视范围”与“agent 物理半径”同时作为空间分层依据：

   a. 3 m 圈—主图（长期记忆）：用原有方式追加式更新，不删除历史，保证全局一致性。

   b. 1 m 圈—子图（短时记忆）：每次帧到先清空再重画，只反映当前瞬间的局部环境，随后通过“匹配-融合-写回”三步把可信变化同步到主图。

- 存在的问题：

    - 需要维护两张地图 + 一次双向同步，工程实现复杂度翻倍。
    - 匹配-融合阶段若位姿漂移，会把错误写回主图，形成障碍地图错位以及语义信息错误。
2. 基于场景变化的地图更新
    a. 先做一次"快照"：把初始可见范围里的障碍+语义整体存成"参考帧"，并按照区域进行划分。
    b. 此后每来一帧，只在同一范围内做"变化评分"，分数超过阈值才把整个范围标记为"待更新"，并用新观测整体替换旧内容；否则跳过，保持主图不动。
- 存在的问题：
    - 对于评价函数的要求较高，场景中可能会出现语义无变化但障碍地图变化大的情况
3. 基于任务预测的地图更新
    a. 当前VLN每次推理完成时将会提供一个目标点，由agent到目标点的最短路径结合机器人可视范围进行更新范围选取并进行更新；
        i. 更新范围选取规则：结合最短路径以及机器人正方向视场（锥形）绘制

待办：

json文件+障碍地图贴图效率

障碍地图使用导航处提供的地图

语义标签更新的策略

结合JQR设备

当前VLN方案架构图

效果、输入、围绕